

Build "Chat With Your Docs" — A RAG Portfolio Project

A step-by-step guide to building a Retrieval-Augmented Generation app with source citations, using Python, ChromaDB, and the Claude API.

0. Setup

Create a project folder and virtual environment.

```
mkdir rag-project && cd rag-project
python3 -m venv venv
source venv/bin/activate # on Windows: venv\Scripts\activate
```

```
pip install pinecone anthropic streamlit pypdf tiktoken python-dotenv
```

Get an Anthropic API key from the [Claude Console](#). You already have a Pinecone account and API key. Create a `.env` file:

```
ANTHROPIC_API_KEY=your_anthropic_key_here
PINECONE_API_KEY=your_pinecone_key_here
```

Your folder structure will look like this by the end:

```
rag-project/
├── .env
├── ingest.py    # Step 1-2: load docs, chunk, embed, store
├── retrieve.py  # Step 3: search the vector DB
├── generate.py  # Step 4: call Claude with retrieved context
├── app.py      # Step 5: Streamlit UI
└── docs/       # put your PDFs / text files here
```

Step 1: Load and chunk your documents

Chunking matters a lot for RAG quality. Too big and retrieval is imprecise; too small and you lose context. We'll use ~500-token chunks with overlap.

```
# ingest.py
import os
from pypdf import PdfReader
import tiktoken

DOCS_DIR = "docs"
CHUNK_SIZE = 500    # tokens per chunk
CHUNK_OVERLAP = 50  # tokens shared between consecutive chunks

encoding = tiktoken.get_encoding("cl100k_base")

def load_documents(docs_dir):
    """Read all .txt and .pdf files in docs_dir, return list of (filename, text)."""
    documents = []
    for filename in os.listdir(docs_dir):
        path = os.path.join(docs_dir, filename)
        if filename.endswith(".txt"):
            with open(path, "r", encoding="utf-8") as f:
                documents.append((filename, f.read()))
        elif filename.endswith(".pdf"):
            reader = PdfReader(path)
            text = "\n".join(page.extract_text() or "" for page in reader.pages)
            documents.append((filename, text))
    return documents

def chunk_text(text, chunk_size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    """Split text into overlapping chunks measured in tokens."""
    tokens = encoding.encode(text)
    chunks = []
    start = 0
    while start < len(tokens):
        end = start + chunk_size
        chunk_tokens = tokens[start:end]
        chunks.append(encoding.decode(chunk_tokens))
        start += chunk_size - overlap
    return chunks

def build_chunks():
```

```

"""Return a list of dicts: {id, text, source, chunk_index}."""
documents = load_documents(DOCS_DIR)
all_chunks = []
for filename, text in documents:
    chunks = chunk_text(text)
    for i, chunk in enumerate(chunks):
        all_chunks.append({
            "id": f"{filename}-{i}",
            "text": chunk,
            "source": filename,
            "chunk_index": i,
        })
    return all_chunks

if __name__ == "__main__":
    chunks = build_chunks()
    print(f"Loaded {len(chunks)} chunks from {DOCS_DIR}")
    print("Example chunk:\n", chunks[0]["text"][:300] if chunks else "No documents found")

```

Drop a few `.txt` or `.pdf` files into `docs/`, then run:

```
python ingest.py
```

You should see the chunk count printed. **This is your first checkpoint — don't move on until this works.**

Step 2: Embed chunks and store them in Pinecone

We'll use Pinecone's **integrated inference** — it embeds your text server-side using a hosted model, so you don't need a separate embeddings provider or API key. We'll use `multilingual-e5-large` (1024 dimensions, strong general-purpose quality).

```
# ingest.py (add to the bottom, below build_chunks)
```

```

import os
import time
from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec

```

```
load_dotenv()
```

```
INDEX_NAME = "rag-docs"
```

```
EMBED_MODEL = "multilingual-e5-large"
```

```
EMBED_DIM = 1024
```

```
BATCH_SIZE = 90 # Pinecone's inference API caps batch size around 96 texts
```

```
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
```

```
def get_index():
```

```
    if not pc.has_index(INDEX_NAME):
```

```
        pc.create_index(
```

```
            name=INDEX_NAME,
```

```
            dimension=EMBED_DIM,
```

```
            metric="cosine",
```

```
            spec=ServerlessSpec(cloud="aws", region="us-east-1"),
```

```
        )
```

```
        while not pc.describe_index(INDEX_NAME).status["ready"]:
```

```
            time.sleep(1)
```

```
    return pc.Index(INDEX_NAME)
```

```
def embed_texts(texts, input_type="passage"):
```

```
    """input_type is 'passage' when embedding docs, 'query' when embedding a search query."""
```

```
    result = pc.inference.embed(
```

```
        model=EMBED_MODEL,
```

```
        inputs=texts,
```

```
        parameters={"input_type": input_type, "truncate": "END"},
```

```
    )
```

```
    return [r["values"] for r in result]
```

```
def embed_and_store(chunks):
```

```
    index = get_index()
```

```
    for i in range(0, len(chunks), BATCH_SIZE):
```

```
        batch = chunks[i:i + BATCH_SIZE]
```

```
        vectors = embed_texts([c["text"] for c in batch], input_type="passage")
```

```
        upserts = [
```

```
            {
```

```
                "id": c["id"],
```

```
                "values": vec,
```

```
                "metadata": {"source": c["source"], "chunk_index": c["chunk_index"], "text": c["text"]},
```

```
            }
```

```

        for c, vec in zip(batch, vectors)
    ]
    index.upsert(vectors=upserts)
    print(f"Upserted batch {i // BATCH_SIZE + 1} ({len(batch)} chunks)")
    print(f"Stored {len(chunks)} chunks in Pinecone index '{INDEX_NAME}'")

```

```

if __name__ == "__main__":
    chunks = build_chunks()
    print(f"Loaded {len(chunks)} chunks from {DOCS_DIR}")
    embed_and_store(chunks)

```

Run it again:

```
python ingest.py
```

The first run creates the index in your Pinecone account (check the Pinecone console — you'll see it there) and can take a few seconds to spin up. You only need to re-run this when your documents change.

Note: we store the raw chunk `text` inside Pinecone's metadata so we can retrieve it directly in Step 3 without a separate lookup table. Fine for a portfolio project; for very large corpora you'd store text elsewhere and keep Pinecone metadata small.

Step 3: Retrieval — semantic search over your chunks

```

# retrieve.py
from ingest import get_index, embed_texts

def search(query, top_k=4):
    """Return the top_k most relevant chunks for a query, with sources."""
    index = get_index()
    query_vector = embed_texts([query], input_type="query")[0]

    results = index.query(
        vector=query_vector,
        top_k=top_k,
        include_metadata=True,
    )

```

```

hits = []
for match in results["matches"]:
    meta = match["metadata"]
    hits.append({
        "text": meta["text"],
        "source": meta["source"],
        "chunk_index": meta["chunk_index"],
        "score": match["score"], # cosine similarity, higher = more relevant
    })
return hits

```

```

if __name__ == "__main__":
    query = input("Test query: ")
    for hit in search(query):
        print(f"\n[{hit['source']}] chunk {hit['chunk_index']} (score={hit['score']:.3f})")
        print(hit["text"][:200])

```

Test it:

python retrieve.py

Type a question related to your docs and confirm the chunks that come back are actually relevant. **If retrieval is bad, no amount of prompt engineering will fix the final answer — so verify this step carefully before moving on.**

Step 4: Generation — send retrieved context to Claude

This is where source citations come in. We number each chunk and instruct the model to cite which chunk(s) it used.

```

# generate.py
import os
from dotenv import load_dotenv
import anthropic
from retrieve import search

load_dotenv()
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

```

SYSTEM_PROMPT = """"You are a helpful assistant that answers questions using ONLY the provided document excerpts. Each excerpt is numbered.

Rules:

- If the answer isn't in the excerpts, say so clearly — do not make things up.
- After each claim, cite the excerpt number(s) it came from, like [1] or [1,3].
- Be concise and direct.

""""

```
def build_context_block(hits):
    lines = []
    for i, hit in enumerate(hits, start=1):
        lines.append(f"[{i}] (source: {hit['source']}, chunk {hit['chunk_index']})\n{hit['text']}")
    return "\n\n".join(lines)
```

```
def answer_question(question, top_k=4):
    hits = search(question, top_k=top_k)
    context = build_context_block(hits)
```

```
    user_message = f""""Document excerpts:
```

```
{context}
```

```
Question: {question}""""
```

```
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=800,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": user_message}],
    )
```

```
    answer_text = "".join(block.text for block in response.content if block.type == "text")
    return answer_text, hits
```

```
if __name__ == "__main__":
    question = input("Ask a question about your docs: ")
    answer, hits = answer_question(question)
    print("\n--- ANSWER ---")
    print(answer)
```

```

print("\n--- SOURCES ---")
for i, hit in enumerate(hits, start=1):
    print(f"[{i}] {hit['source']} (chunk {hit['chunk_index']})")

```

Run it:

```
python generate.py
```

You now have a working RAG pipeline end to end. **This alone is demo-able** — but let's make it look like a real product.

Step 5: A simple Streamlit UI

```

# app.py
import streamlit as st
from generate import answer_question

st.set_page_config(page_title="Chat With Your Docs", page_icon="📄")
st.title("📄 Chat With Your Docs")
st.caption("A RAG app that answers questions from your document collection, with citations.")

if "history" not in st.session_state:
    st.session_state.history = []

question = st.chat_input("Ask a question about your documents...")

for entry in st.session_state.history:
    with st.chat_message("user"):
        st.write(entry["question"])
    with st.chat_message("assistant"):
        st.write(entry["answer"])
    with st.expander("Sources"):
        for i, hit in enumerate(entry["hits"], start=1):
            st.markdown(f"**[{i}] {hit['source']}** (chunk {hit['chunk_index']})")
            st.text(hit["text"][:300] + "...")

if question:
    with st.chat_message("user"):
        st.write(question)

    with st.spinner("Searching documents and generating answer..."):

```

```

answer, hits = answer_question(question)

with st.chat_message("assistant"):
    st.write(answer)
    with st.expander("Sources"):
        for i, hit in enumerate(hits, start=1):
            st.markdown(f"***[{i}] {hit['source']}** (chunk {hit['chunk_index']})")
            st.text(hit["text"][:300] + "...")

st.session_state.history.append({"question": question, "answer": answer, "hits": hits})

```

Run it:

```
streamlit run app.py
```

You should get a chat interface in your browser at localhost:8501.

Step 6 (the differentiator): A tiny evaluation set

This is the part most beginner portfolios skip, and it's a strong signal to employers that you think about quality, not just demos.

```

# eval.py
from generate import answer_question

# Write 5-10 question/expected-source pairs based on your own docs
EVAL_SET = [
    {"question": "What is the refund policy?", "expected_source": "policy.pdf"},
    {"question": "Who is the CEO mentioned in the report?", "expected_source": "report.txt"},
    # add more based on your actual documents
]

def run_eval():
    correct = 0
    for case in EVAL_SET:
        answer, hits = answer_question(case["question"])
        retrieved_sources = [h["source"] for h in hits]
        hit_found = case["expected_source"] in retrieved_sources
        correct += hit_found

```

```

print(f"Q: {case['question']}")
print(f" Expected source found in top-k: {hit_found}")
print(f" Answer: {answer[:150]}...\n")

print(f"Retrieval accuracy: {correct}/{len(EVAL_SET)}")

if __name__ == "__main__":
    run_eval()

```

Write up the results in your README — e.g., "Achieved 8/10 retrieval accuracy on a hand-built eval set; the 2 misses were due to chunk size splitting a key sentence across boundaries, which I fixed by increasing overlap to 100 tokens." **That sentence alone is worth more on a resume than the whole rest of the project** — it shows iteration and real engineering judgment.

Step 7: Deploy it

- Push to GitHub with a clear [README.md](#) (problem, architecture diagram, tech stack, what you'd improve next).
- Deploy free via [Streamlit Community Cloud](#) — connect your GitHub repo, add your API key as a secret, done.
- Add the live link + repo link to your resume/LinkedIn.

What to write in your README (this matters as much as the code)

1. One-paragraph problem statement
 2. Architecture diagram (ingestion → embedding → vector store → retrieval → generation)
 3. Key design decisions (chunk size, top-k, why Chroma, why Claude)
 4. Your eval results and what you learned from them
 5. "Next steps" section — shows you understand this isn't "finished," which is realistic and honest
-

Where to go from here

Once this works, good next additions (pick one, don't do all at once):

- **Reranking:** add a reranker step after retrieval to improve precision
- **Streaming responses:** use Claude's streaming API so answers appear token-by-token
- **Multi-turn memory:** let follow-up questions reference earlier answers
- **Agentic behavior:** let the model decide whether to search docs, calculate something, or ask a clarifying question first